



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

ИУ «Информатика и системы управления»

КАФЕДРА

ИУ-1 «Системы автоматического управления»

Лабораторная работа № 2
«Бинаризация и математическая морфология»
по курсу «Распознавание образов»

Вариант 6

Студент

ИУ1-31М

(Группа)

(Подпись, дата)

Р.В. Садовец

(И.О. Фамилия)

Проверил

(Подпись, дата)

В.С. Водопьянов

(И.О. Фамилия)

2026 г.

1. Бинаризация, фильтрация, разметка

Рассмотрим изображение риса (рисунок 1).

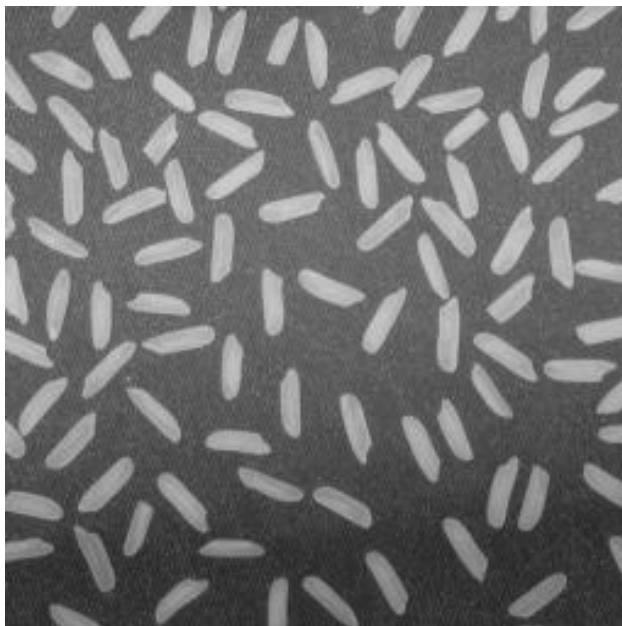


Рисунок 1 — Исходное изображение риса

Проведём фильтрацию и бинаризацию изображения. Для устранения влияния разнотонности фона, вычтем из исходного изображения «карту фона», полученную путём размывания (эрозия+делатация) с кругом радиусом 15 пикселей. Далее произведём бинаризацию посредством порога Оцу и удаления мелких объектов функциями из библиотеки `skimage` в Python:

```
footprint = morphology.disk(15)
background = morphology.opening(gray_image, footprint)

image_clean = gray_image.astype(np.int16) -
background.astype(np.int16)

threshold = filters.threshold_otsu(image_clean)
binary_image = image_clean > threshold

binary_filtered = morphology.remove_small_objects(binary_image,
min_size=30)
```

Результаты фильтрации, бинаризации и удаления мелких объектов представлены на рисунке 2. Также на рисунке отражен порог Оцу, который получился равным 52.

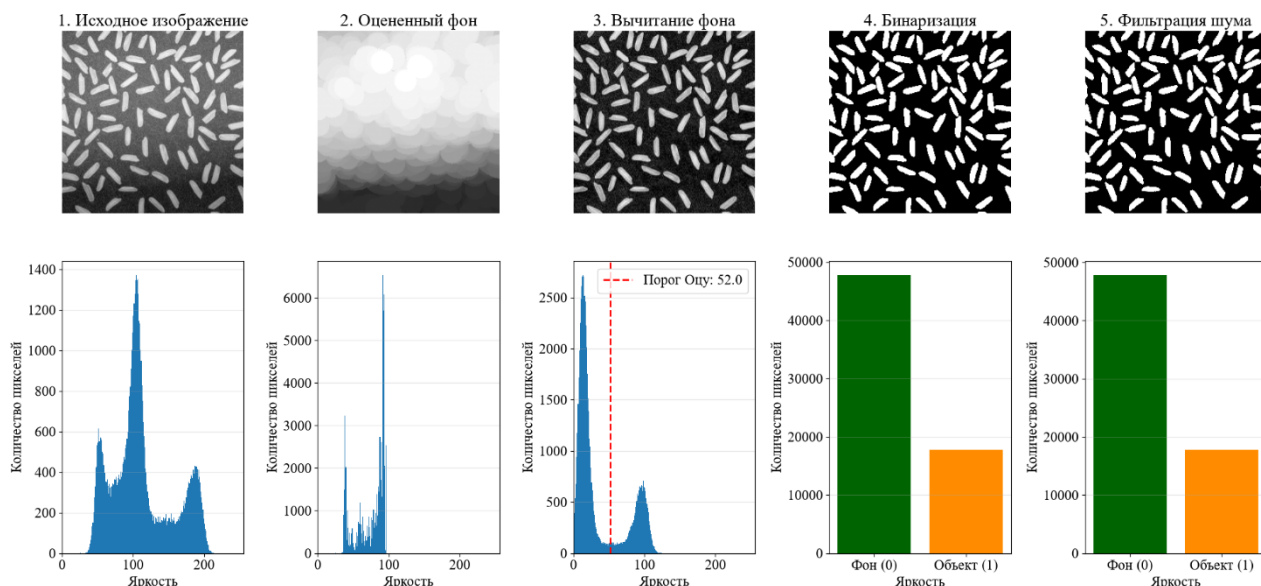


Рисунок 2 — Бинаризация и фильтрация с исходного изображения

Для разметки, поиска целевого зерна и подсчёта суммарного количества зёрен воспользуемся библиотекой `skimage`. Ниже представлен программный код на языке программирования Python. Для распознавания склеившихся зёрен будем придерживаться следующей логики: если площадь размеченного зерна больше 1.8 от средней площади разметки, то это склеенные зерна.

```
labeled_image, total_grains = measure.label(binary_filtered,
return_num=True)

grain_mask =(labeled_image == TARGET_GRAIN_ID)# = 30

if np.any(grain_mask):
    isolated_grain = np.where(grain_mask, gray_image, 0)
    print(f"Зерно номер {TARGET_GRAIN_ID} успешно найдено.")
else:
    print(f"Зерно с номером {TARGET_GRAIN_ID} не найдено.")
    isolated_grain = np.zeros_like(gray_image)

properties = measure.regionprops(labeled_image)
areas = [prop.area for prop in properties]
mean_area = np.mean(areas)

# Склеившиеся зерна: площадь > 1.8 от средней
clumped_threshold = mean_area * 1.8
clumped_grains = [prop.label for prop in properties if prop.area
> clumped_threshold]
```

Тогда получим следующий результат:

Всего зёрен риса на изображении: 94
Зерно номер 30 успешно найдено.
Склеившиеся зерна: 3 шт.
Номера: [15, 46, 47]

На рисунке 3 представлена полученная разметка, а также отображение целевого и склеившихся зёрен.

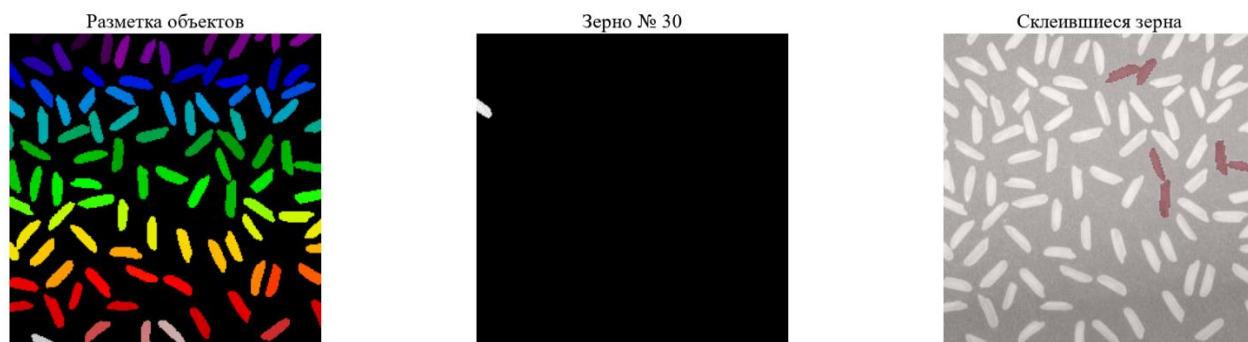


Рисунок 3 — Разметка риса на изображении

2. Характеристики бинарных объектов

Рассмотрим изображение с несколькими деталями (рисунок 4). Необходимо распознать деталь с четырьмя отверстиями, а также пятиугольную деталь.

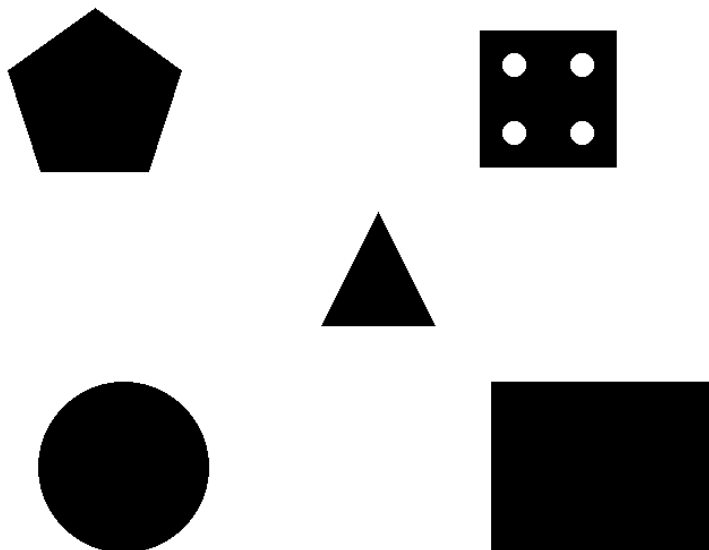


Рисунок 4 — Исходная картинка с фигурами

Ниже представлен программный код для поиска детали с отверстиями. Для этого строится маска с отверстиями, после чего по ней определяется принадлежности определенной фигуре.

```
filled_objects = ndimage.binary_fill_holes(binary) # Изображение
с заполненными отверстиями
holes_mask = filled_objects.astype(np.uint8) - binary

# Разметка
labeled_input = measure.label(binary) # Исходное изображение
labeled_filled = measure.label(filled_objects) # Изображение с
заполненными отверстиями внутри деталей
labeled_holes, num_holes_total = measure.label(holes_mask,
return_num=True) # Изображение, где ТОЛЬКО отверстия внутри
деталей

# Для каждого отверстия смотрим, внутри какой заполненной детали
он лежит
holes_count = {}
for h in range(1, num_holes_total + 1):
    r, c = np.argwhere(labeled_holes == h)[0]
    obj = labeled_filled[r, c]
    holes_count[obj] = holes_count.get(obj, 0) + 1
```

Для определения же количество углов воспользуемся центроидальным профилем. Центр определяется средним между минимальными и максимальными координатами. Ниже представлена функция для поиска центра и вращения профиля.

```
def centroid_profile(obj_mask, n_bins=360):
    obj_mask = np.asarray(obj_mask, dtype=bool)

    # Границы объекта (аналог bwperim)
    perim = obj_mask & ~ndimage.binary_erosion(obj_mask)
    ys, xs = np.nonzero(perim)

    # Центр масс объекта – среднее координат его точек
    obj_ys, obj_xs = np.nonzero(obj_mask)
    xc, yc = obj_xs.mean(), obj_ys.mean()

    # Длина и направление радиус-вектора каждой точки границы
    ro = np.hypot(xs - xc, ys - yc)
    fi = np.arctan2(ys - yc, xs - xc) # [-pi, pi]

    # R[fi]
    idx = ((fi + np.pi) / (2 * np.pi) *
n_bins).astype(int).clip(0, n_bins - 1)
    R = np.zeros(n_bins)
    cnt = np.zeros(n_bins)
    np.add.at(R, idx, ro)
    np.add.at(cnt, idx, 1)
    nonempty = np.nonzero(cnt)[0]
    R[nonempty] /= cnt[nonempty]
    ext_idx = np.concatenate([nonempty[-1:] - n_bins, nonempty,
nonempty[:1] + n_bins])
    ext_val = np.concatenate([R[nonempty][-1:], R[nonempty],
R[nonempty][:1]])
    R = np.interp(np.arange(n_bins), ext_idx, ext_val)

    return R / R.mean()
```

На рисунке 5 показаны результаты построения центроидального профиля для всех фигур. По ним явным образом можно определить, какое количество вершин у детали, за исключением окружности.

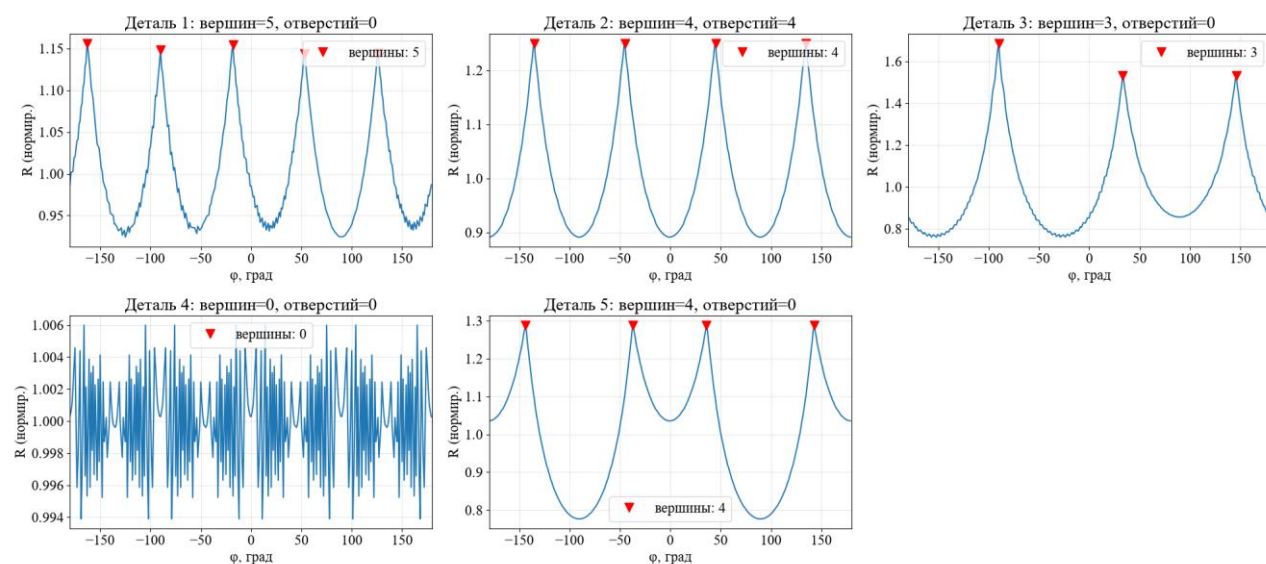


Рисунок 5 — Центроидальный профиль размеченных фигур

Результаты разметки пятиугольника и поиска детали с четырьмя отверстиями представлен на рисунке 6.

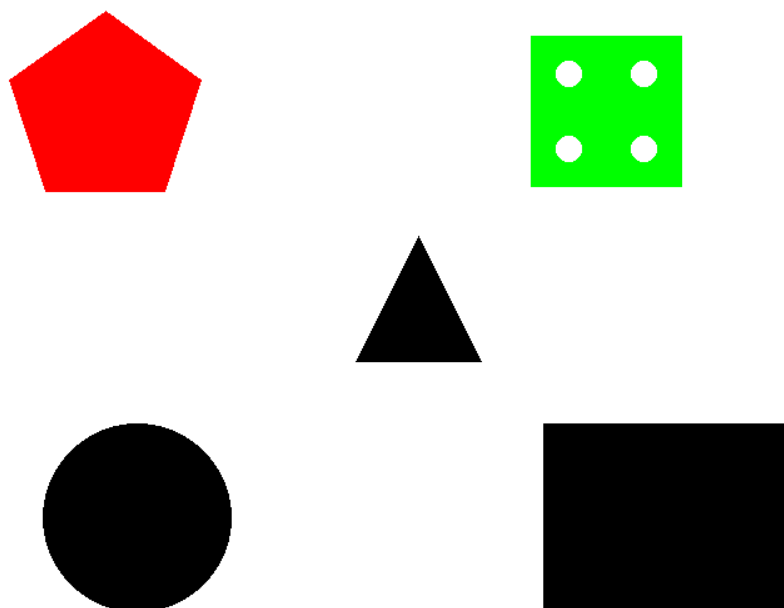


Рисунок 6 — Итоговая разметка фигур